

Module 7

Version Comparison and Migration

Learning objectives

- Compare Verilog and SystemVerilog across IEEE versions side-by-side, plan migration between standar...

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Side-by-Side Comparison: Sam"]

T1 --> N["Next module in course"]

Compare Verilog and SystemVerilog across IEEE versions side-by-side, plan migration between standards, and choose the right version for...

Overview

- This module ties together Modules 1–6 by comparing the same design across standards (1364-1995 thro...

How to learn this module

Suggested learning path (1/2)

- Skim this document — Goal, Overview, and Topics
Covered set the IEEE scope for Module 7.
- Study design architecture — See how DUTs, examples, and testbenches fit together under ``module7/``.
- Work through labs in order — Open ``module7/EXAMPLES.md`` and run each ``make clean && make run`` from...
- Run the full module script — ``./scripts/module7.sh`` from the repo root to simulate all examples and...
- Complete the exercises — Try each exercise in this document before reading solutions or peeking at...

Suggested learning path (2/2)

- Compare side-by-side — Open 1364 and 1800 versions of the same design and note port style, types, a...
- Review Common Pitfalls — Can you explain each mistake and the fix?
- Self-check — Use ``module7/CHECKLIST.md`` before starting the next module.

Design architecture

1. Repository layout (module7/) (1/2)

- Side-by-side 1364 vs 1800 versions of the same designs
- module7/dut/ — mux2_1364.v and mux2_1800.sv for direct comparison
- examples/side_by_side/ — decoder, counter, FSM migration pairs
- examples/migration/ — step-by-step 1995 → 2017 patterns

```
Rtl Architecture
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
subgraph PAIR["Same behavior — two syntaxes"]
M14["mux2_1364.v"]
M18["mux2_1800.sv"]
C14["counter_1364.v"]
C18["counter_1800.sv"]
```

1. Repository layout (module7/) (2/2)

- scripts/module7.sh — runs comparison and migration
 labs

2. Side-by-side comparison architecture

- Same behavior, different syntax
 - ports, types, procedural blocks
- 1364: wire/reg, always @(a or b), many scalar ports
- 1800: logic, always_comb/ff, optional interface for bus bundles
- Testbench can instantiate both versions and compare outputs

Verification Architecture

```
Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR
    TB["test_mux2_all.sv"] --> M14
    TB --> M18
    CMP["output compare"] --> PASS["equivalent"]
    M14 --> CMP
    M18 --> CMP
```

3. Migration workflow

- Inventory features used — match to target standard checklist
- Migrate ports first (ANSI, logic), then procedural blocks, then connectivity
- Simulate after each step — do not big-bang rewrite
- Document project standard and tool support decision

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TB
  M7["module7.sh"] --> MUX["test_mux2_all cross-version"]
  M7 --> CTR["test_counter_all"]
  MUX --> MIG["migration checklist tick"]
  CTR --> MIG
```

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph PAIR["Same behavior — two syntaxes"]

M14["mux2_1364.v"]

M18["mux2_1800.sv"]

C14["counter_1364.v"]

C18["counter_1800.sv"]

Module 7: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

TB["test_mux2_all.sv"] --> M14

TB --> M18

CMP["output compare"] --> PASS["equivalent"]

M14 --> CMP

M18 --> CMP

Module 7: stimulus, observation, and checking.

1364 mux — wire/reg and explicit sensitivity

```
module mux2_1995(a, b, sel, y);  
    input  a, b, sel;  
    output y;  
    wire   a, b, sel;  
    reg    y;  
    always @(a or b or sel)  
        y = sel ? b : a;  
endmodule
```

1800 mux — logic and always_comb

```
module mux2_1800 (  
    input  logic a, b, sel,  
    output logic y  
);  
    always_comb y = sel ? b : a;  
endmodule
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator compiles RTL + SystemVerilog testbench into a C++ model
- sim_main.cpp: generates clk/rst_n, calls eval() until \$finish
- Directed test (initial block or C++): drive stimulus, wait for DUT flags
- Self-check: compare outputs; print PASS/FAIL (see terminal demo slide)
- Repo path: module7/examples/<example>/ — run make run from that directory

Key files to study

Open these in the repo

- `module7/dut/mux2_1364.v` and `mux2_1800.sv`
- `module7/examples/side_by_side/mux2_versions/`
- `module7/examples/migration_checklist/`
- `module7/examples/incremental_migration/`
- `scripts/module7.sh`

Verification & testing methods

1. Comparison test execution

- test_mux2_all.sv — same stimulus on 1364 and 1800 mux; compare y
- test_counter_all.sv — reset/enable/wrap behavior must match
- ./scripts/module7.sh runs all migration and comparison examples

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

M7["module7.sh"] --> MUX["test_mux2_all cross-version"]

M7 --> CTR["test_counter_all"]

MUX --> MIG["migration checklist tick"]

CTR --> MIG

2. Migration validation

- After each migration step: make run must pass with identical outputs
- Use migration_checklist example as a template for real projects

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

M7["module7.sh"] --> MUX["test_mux2_all cross-version"]

M7 --> CTR["test_counter_all"]

MUX --> MIG["migration checklist tick"]

CTR --> MIG

Migration checklist pattern

```
/*
 * Module 7: Migration checklist reminder (1364 -> 1800 design subset)
 * Prints steps; full checklist in MODULE7.md.
 */

module top;
    initial begin
        $display("Module 7: Migration checklist (1364 -> 1800 design
subset)");
        $display("  Pre: target standard, baseline sim/synth, list
files/hierarchy");
        $display("  Per-module: ports wire/reg->logic, @*->always_comb,
posedge clk->always_ff");
        $display("  Per-module: case->unique/priority case, optional
interface/package, no defparam");
        $display("  Post: full regression, update style guide, document
1364-only blocks");
```

Syllabus topics

Protocol & design details

1. Side-by-Side Comparison: Same Design Across Versions

- Example: 2:1 Multiplexer
- What Changes Across Versions

Hands-on examples

Module 7 toolchain check

```
$ command -v iverilog >/dev/null && echo "Toolchain ready: $(iverilog  
-V 2>&1 | head -1)"
```

Terminal: module7_check

Toolchain ready: Icarus Verilog version 12.0 (stable) ()

Example 1: Same 2:1 mux in 1364-1995, 1364-2001, 1364-2005, 1800-2005...

- Port style, type, procedural block, one driver

Demo: Same 2:1 mux in 1364-1995, 1364-2001, 1364-2005, 1800-2005 (and...

```
$ cd module7/examples/side_by_side && make clean && make run
```

Terminal: ex01_side_by_side

make: *** No rule to make target 'clean'. Stop.

Example 2: Same counter in 1995, 2001, 2005, 1800-2005

- Sequential block, reset, parameter/localparam

Demo: Same counter in 1995, 2001, 2005, 1800-2005

```
$ cd module7/examples/side_by_side && make clean && make run
```

Terminal: ex02_side_by_side

make: *** No rule to make target 'clean'. Stop.

Example 3: Same 2:4 decoder in 1995, 2001, 2005, 1800; case vs unique...

- Port style, always @(sel) vs @* vs always_comb, unique case

Demo: Same 2:4 decoder in 1995, 2001, 2005, 1800; case vs unique case

```
$ cd module7/examples/side_by_side && make clean && make run
```

Terminal: ex03_side_by_side

make: *** No rule to make target 'clean'. Stop.

Example 4: Same 8-bit adder in 1995, 2001, 2005, 1800 (continuous ass...

- wire vs logic; assign unchanged across versions

Demo: Same 8-bit adder in 1995, 2001, 2005, 1800 (continuous assign)

```
$ cd module7/examples/side_by_side && make clean && make run
```

Terminal: ex04_side_by_side

make: *** No rule to make target 'clean'. Stop.

Example 5: Same parameterized shift in 1364 (parameter/localparam) vs...

- parameter int, localparam int; same behavior

Demo: Same parameterized shift in 1364 (parameter/localparam) vs 1800...

```
$ cd module7/examples/side_by_side && make clean && make run
```

Terminal: ex05_side_by_side

make: *** No rule to make target 'clean'. Stop.

Example 6: Same master/slave connection: many ports (1364)

- Port list size, direction, reuse; interface (1800)
optional per tool

Demo: Same master/slave connection: many ports (1364)

```
$ cd module7/examples/side_by_side && make clean && make run
```

Terminal: ex06_side_by_side

make: *** No rule to make target 'clean'. Stop.

Example 7: Before/after: wire/reg, always @* vs logic, always_comb, u...

- Order of steps, testing, regression

Demo: Before/after: wire/reg, always @* vs logic, always_comb, unique...

```
$ cd module7/examples/migration && make clean && make run
```

Terminal: ex07_migration

```
rm -f top
iverilog -g2012 -o top migration_steps.v
vvp top
Module 7: Migration 1364 -> 1800 (steps: wire/reg->logic, @*->always_comb, unique case)
  sel=10 y_before=04 y_after=04
  PASS: same behavior
migration_steps.v:53: $finish called at 10 (1s)
migration_steps.v:30: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Example 8: Before: non-ANSI, @(a or b or sel). After: ANSI, always @*

- ANSI ports, implicit sensitivity

Demo: Before: non-ANSI, @(a or b or sel). After: ANSI, always @*

```
$ cd module7/examples/migration_1995_to_2001 && make clean && make run
```

```
Terminal: ex08_migration_1995_to_2001  
rm -f top  
iverilog -o top migration_1995_2001.v  
vvp top  
Module 7: Migration 1995 -> 2001 (ANSI ports, always @*)  
  sel=0: y1995=0 y2001=0  
  sel=1: y1995=1 y2001=1  
  PASS: same behavior  
migration_1995_2001.v:40: $finish called at 20 (1s)
```

Example 9: Same 4:1 mux: 1364 case+default vs 1800 unique case

- case semantics; unique case for tool checking

Demo: Same 4:1 mux: 1364 case+default vs 1800 unique case

```
$ cd module7/examples/case_versions && make clean && make run
```

Terminal: ex09_case_versions

```
rm -f top
iverilog -g2012 -o top case_versions.sv
vvp top
Module 7: case 1364 (case+default) vs 1800 (unique case)
  sel=01 y1364=02 y1800=02
  sel=11 y1364=08 y1800=08
  PASS: same behavior
case_versions.sv:54: $finish called at 20 (1s)
case_versions.sv:30: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Example 10: Single reference: construct vs standard (1995-2017)

- When each feature was introduced; minimum revision

Demo: Single reference: construct vs standard (1995-2017)

```
$ cd module7/examples/version_table && make clean && make run
```

Terminal: ex10_version_table

```
rm -f top
iverilog -g2012 -o top version_table.sv
vvp top
Module 7: Version table (construct vs standard)
Port style: 1995 non-ANSI | 2001+ ANSI
Type:      1364 wire/reg | 1800 logic (single driver)
Combinational: 1995 @(a or b) | 2001 @* | 1800 always_comb
Sequential: 1364 always @(posedge clk) | 1800 always_ff
Connectivity: 1364 many ports | 1800 interface+modport
Case:      1364 case | 1800 unique/priority case
Reference: 1800-2017 single LRM (Verilog + SystemVerilog)
version_table.sv:16: $finish called at 0 (1s)
```

Example 11: Runnable reminder of version-selection checklist (constra...

- See MODULE7.md for full checklist

Demo: Runnable reminder of version-selection checklist (constraints,...

```
$ cd module7/examples/version_selection && make clean && make run
```

Terminal: ex11_version_selection

```
rm -f top
iverilog -g2012 -o top version_selection.sv
vvp top
Module 7: Version selection checklist (reminder)
  Constraints: tools (sim/synth), IP/legacy, team, target (FPGA/ASIC)
  Verilog-only (1364): wire/reg, always @*, no logic/interfaces/packages
  SystemVerilog design (1800): logic, always_comb/always_ff, packages, unique/priority case
  Decision: state one revision (e.g. IEEE 1800-2017); define subset if needed
  Document: style guide or README; reference LRM and tool manuals
  See MODULE7.md for full version-selection checklist.
version_selection.sv:15: $finish called at 0 (1s)
```

Example 12: Same D flip-flop in 1995, 2001, 2005, 1800 (sequential bl...

- always @(posedge clk) vs always_ff; wire/reg vs logic

Demo: Same D flip-flop in 1995, 2001, 2005, 1800 (sequential block st...

```
$ cd module7/examples/side_by_side && make clean && make run
```

Terminal: ex12_side_by_side

make: *** No rule to make target 'clean'. Stop.

Example 13: Same tiny FSM in 1364 vs 1800 (case vs unique case, alway...

- Sequential FSM; 1800 always_ff and unique case

Demo: Same tiny FSM in 1364 vs 1800 (case vs unique case, always_ff)

```
$ cd module7/examples/side_by_side && make clean && make run
```

Terminal: ex13_side_by_side

make: *** No rule to make target 'clean'. Stop.

Example 14: Runnable reminder of 1364→1800 migration steps (pre, per-...

- See MODULE7.md for full migration checklist

Demo: Runnable reminder of 1364→1800 migration steps (pre, per-module...

```
$ cd module7/examples/migration_checklist && make clean && make run
```

Terminal: ex14_migration_checklist

```
rm -f top
iverilog -g2012 -o top migration_checklist.sv
vvp top
Module 7: Migration checklist (1364 -> 1800 design subset)
  Pre: target standard, baseline sim/synth, list files/hierarchy
  Per-module: ports wire/reg->logic, @*->always_comb, posedge clk->always_ff
  Per-module: case->unique/priority case, optional interface/package, no defparam
  Post: full regression, update style guide, document 1364-only blocks
  See MODULE7.md for full migration checklist.
migration_checklist.sv:14: $finish called at 0 (1s)
```

Example 15: Two steps: step1 1364 style, step2 1800 style; same mux,...

- Migrate one construct at a time; test after each step

Demo: Two steps: step1 1364 style, step2 1800 style; same mux, compar...

```
$ cd module7/examples/incremental_migration && make clean && make run
```

```
Terminal: ex15_incremental_migration  
rm -f top  
iverilog -g2012 -o top incremental_migration.sv  
vvp top  
Module 7: Incremental migration (step1=1364, step2=1800)  
  sel=11 y_step1=08 y_step2=08  
  PASS: same behavior after migration  
incremental_migration.sv:54: $finish called at 10 (1s)  
incremental_migration.sv:31: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

Example 16: Same 2:1 mux: non-ANSI (1995) vs ANSI (2001) ports only

- Port declaration style; no other change

Demo: Same 2:1 mux: non-ANSI (1995) vs ANSI (2001) ports only

```
$ cd module7/examples/port_style_compare && make clean && make run
```

Terminal: ex16_port_style_compare

```
rm -f top
iverilog -o top port_style_compare.v
vvp top
Module 7: Port style - non-ANSI vs ANSI (same mux)
  sel=0: y_non_ansi=0 y_ansi=0
  sel=1: y_non_ansi=1 y_ansi=1
  PASS: same behavior
port_style_compare.v:39: $finish called at 20 (1s)
```

Example 17: 1364-2005: parameter override at instantiation `#(.WIDTH(n...`

- defparam deprecated; use instantiation override

Demo: 1364-2005: parameter override at instantiation `#(.WIDTH(n)); av...`

```
$ cd module7/examples/no_defparam && make clean && make run
```

Terminal: ex17_no_defparam

```
rm -f top
iverilog -o top no_defparam.v
vvp top
Module 7: No defparam - parameter override at instantiation (1364-2005)
  count8= 5 count16= 5
  Use #(.WIDTH(n)) at instantiation; avoid defparam.
no_defparam.v:36: $finish called at 70 (1s)
```

Common pitfalls

Migrating Everything at Once

- Mistake: Converting a large codebase from 1364 to 1800 in one big change.
- Reality: Hard to debug; regressions are difficult to isolate.
- Correct: Migrate incrementally (one module or one feature at a time); run regression after each step...
- Why: Reduces risk and keeps a clear baseline.

Ignoring Tool Support

- Mistake: Choosing 1800-2017 and using constructs the synthesis or simulator does not support.
- Reality: Compile or elaboration errors; or tool-specific behavior.
- Correct: Check tool manuals for supported revision and synthesizable subset; align RTL with that.
- Why: Ensures the project builds and matches tool expectations.

Breaking Interfaces During Migration

- Mistake: Changing port list or interface of a block while migrating its internals, so that other mo...
- Reality: Large ripple of changes; integration failures.
- Correct: Keep external interface (ports or interface modport) unchanged during internal migration;...
- Why: Limits scope of change and preserves compatibility with rest of design.

Mixing Revisions in One File

- Mistake: Using 1995 ports in one module and 1800 logic in another in the same file without a clear...
- Reality: Confusing for readers and tools; possible tool-dependent behavior.
- Correct: Use one project standard per file (or per project); document the standard in the style gui...
- Why: Consistency and predictable tool behavior.

Forgetting to Test After Migration

- Mistake: Changing syntax (e.g. wire→logic, always @*→always_comb) and not re-running simulation or...
- Reality: Subtle bugs (e.g. latch, wrong sensitivity) can appear.
- Correct: Run full regression (simulation and synthesis) after every migration step; compare with ba...
- Why: Catches functional or synthesis differences early.

Practice & assessment

What you should know

- ✓ Compare the same design across 1364-1995, 1364-2001, 1364-2005, and 1800 (design subset) and list...
- ✓ Compare connectivity style: many ports (1364) vs interface (1800)
- ✓ Plan and execute migration from 1364 to 1800 design subset (or between 1364 revisions) using the...
- ✓ Choose a project standard (revision and subset) based on tool support and constraints
- ✓ Use the version-selection checklist for new or existing projects
- ✓ Describe how to extend the course when a new IEEE 1800 revision is released

Exercises

- Side-by-Side Mux
- Side-by-Side Connectivity
- Migration One Block
- Version Selection
- Version Table

Exercises

- Migrating Everything at Once
- Ignoring Tool Support
- Breaking Interfaces During Migration
- Mixing Revisions in One File
- Forgetting to Test After Migration

Summary & next steps

- Compare Verilog and SystemVerilog across IEEE versions side-by-side, plan migration between standar...
- Complete module7/CHECKLIST.md
- Review module7/EXAMPLES.md and run each lab
- Next: Next module in course